

JAVASCRIPT PROMISES

YouCode
POWERED BY SIMPLON

Aamir El Amiri

SOMMAIRE:

01 INTRODUCTION

02 ASYNCHRONE VS SYNCHRONE

03 CALLBACKS

04 CALLBACKS ASYNCHRONES ET "CALLBACK HELL"

05 LES PROMISES

06 AVANTAGES ET INCONVÉNIENTS DES PROMISES EN JAVASCRIPT

07 CONCLUTION

YouCode
POWERED BY SIMPLON

INTRODUCTION

JavaScript est un langage de programmation monothread et asynchrone.

UN THREAD = UNE PILE D'APPELS =
UNE CHOSE À LA FOIS



YouCode
POWERED BY SIMPLON

ASYNCHRONE VS SYNCHRONE

YouCode
POWERED BY SIMPLON

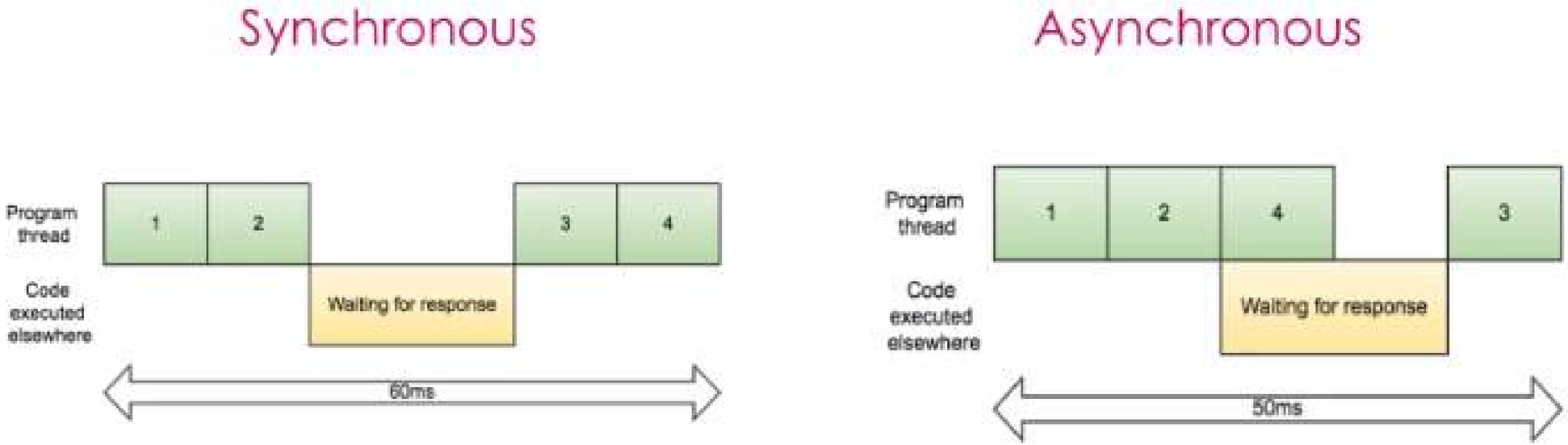
TRAITEMENT SYNCHRONE

Dans les programmes synchrones, si vous avez deux lignes de code (L1 suivie de L2), L2 ne peut pas commencer à s'exécuter tant que L1 n'a pas terminé.

TRAITEMENT ASYNCHRONE

Dans les programmes asynchrones, vous pouvez avoir deux lignes de code (L1 suivie de L2), où L1 planifie une tâche à exécuter dans le futur, mais L2 s'exécute avant que cette tâche ne soit terminée.

ASYNCHRONE VS SYNCHRONE





YouCode
POWERED BY SIMPLON

CALLBACKS

Une fonction de callback est une fonction passée en argument à une autre fonction, qui est ensuite invoquée à l'intérieur de la fonction externe pour accomplir une routine ou une action.

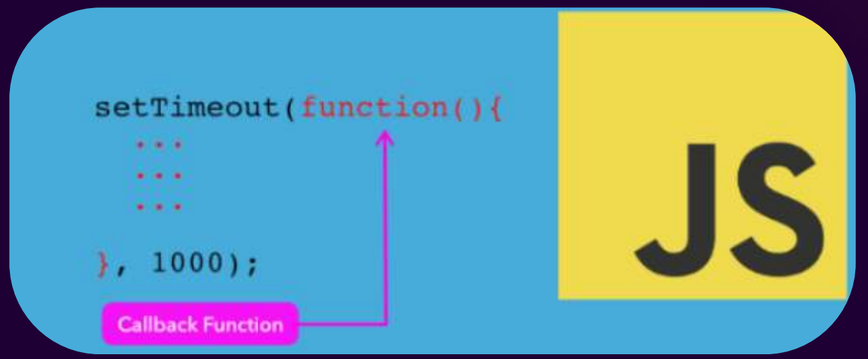
Exemples :

Callback functions in JavaScript

```
function oddOrEven(number, callback) {
    const result =  (number % 2 == 0) ? 'Even' : 'Odd';
    callback(number, result);
}

oddOrEven(28, (number, result) => {
    console.log(number + ' is ' + result);
});

// 28 is Even
```



YouCode
POWERED BY SIMPLON

CALLBACKS ASYNCHRONES ET CALLBACK HELL

CALLBACK ASYNCHRONE

Les callbacks sont souvent utilisés pour continuer l'exécution du code après qu'une opération asynchrone est terminée

CALLBACK HELL

Chaîne d'opérations asynchrones => Code difficile à lire et à déboguer

CALLBACK HELL

```
setTimeout(function(){
                document.getElementById("title1").style.visibility = "visible"

                setTimeout(function(){
                    document.getElementById("title2").style.visibility = "visible"

                    setTimeout(function(){
                        document.getElementById("title3").style.visibility = "visible"

                        setTimeout(function(){
                            document.getElementById("title4").style.visibility = "visible"
                        }, 1000)

                    }, 1000)


                }, 1000)
            }, 1000);
```

YouCode
POWERED BY SIMPLON

YouCode
POWERED BY SIMPLON

LES PROMISES ARRIVENT EN JAVASCRIPT!

Solution pour éliminer le **callback hell**.



QU'EST-CE QU'UNE PROMISE?

Une Promise est un objet qui représente l'achèvement ou l'échec éventuel d'une opération asynchrone et sa valeur résultante.



YouCode
POWERED BY SIMPLON

CONSTRUCTION D'UNE PROMISE

```
new Promise(
    /* executor */
    function(resolve, reject) {
        ...
    }
);
```

01

ON UTILISE **NEW PROMISE** POUR CONSTRUIRE LA PROMISE

02

ON DONNE AU CONSTRUCTEUR **UNE FONCTION EXECUTOR** QUI FAIT LE TRAVAIL RÉEL

03

LA FONCTION EXECUTOR REÇOIT 2 ARGUMENTS: LES FONCTIONS **RESOLVE** ET **REJECT**

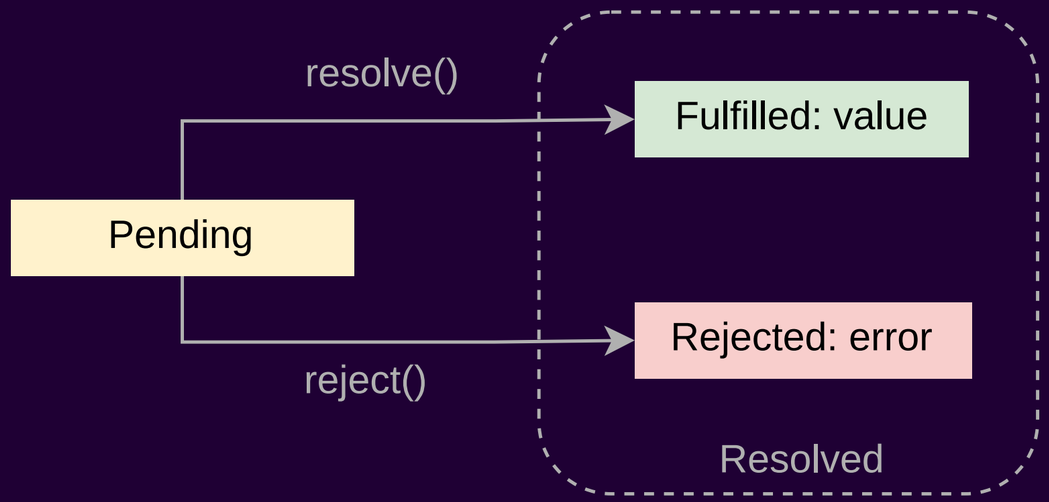
04

LA FONCTION RESOLVE **REMPLIT LA PROMISE** ET LA FONCTION REJECT **LA REJETTE**

ÉTATS D'UNE PROMISE

Une Promise est dans l'un de ces 3 états:

- **pending** : état initial, ni remplie ni rejetée
- **fulfilled** : signifie que l'opération s'est terminée avec succès
- **rejected** : signifie que l'opération a échoué



THEN() ET CATCH()

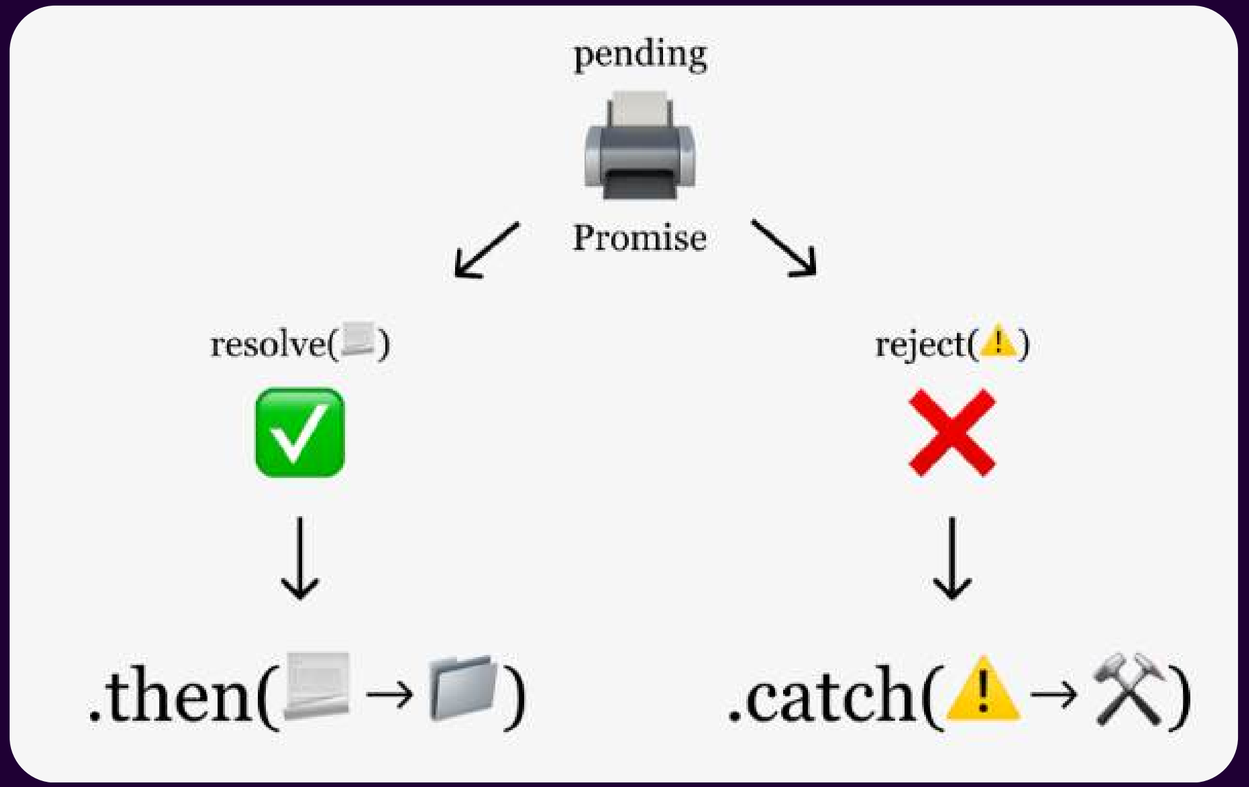
- then() pour récupérer la valeur en cas de succès et .
- catch() pour gérer les erreurs.

```
new Promise((resolve, reject) => {
        setTimeout(() => {
                document.getElementById("title1").style.visibility = "visible"
                resolve()
            }, 1000);
    })
    .then(() => {
        return new Promise((resolve, reject) => {
            setTimeout(() => {
                document.getElementById("title2").style.visibility = "visible"
            }, 1000);
        })
    })
    .then(() => {

        return new Promise((resolve, reject) => {
            setTimeout(() => {
                document.getElementById("title3").style.visibility = "visible"
                resolve()
            }, 1000);
        })

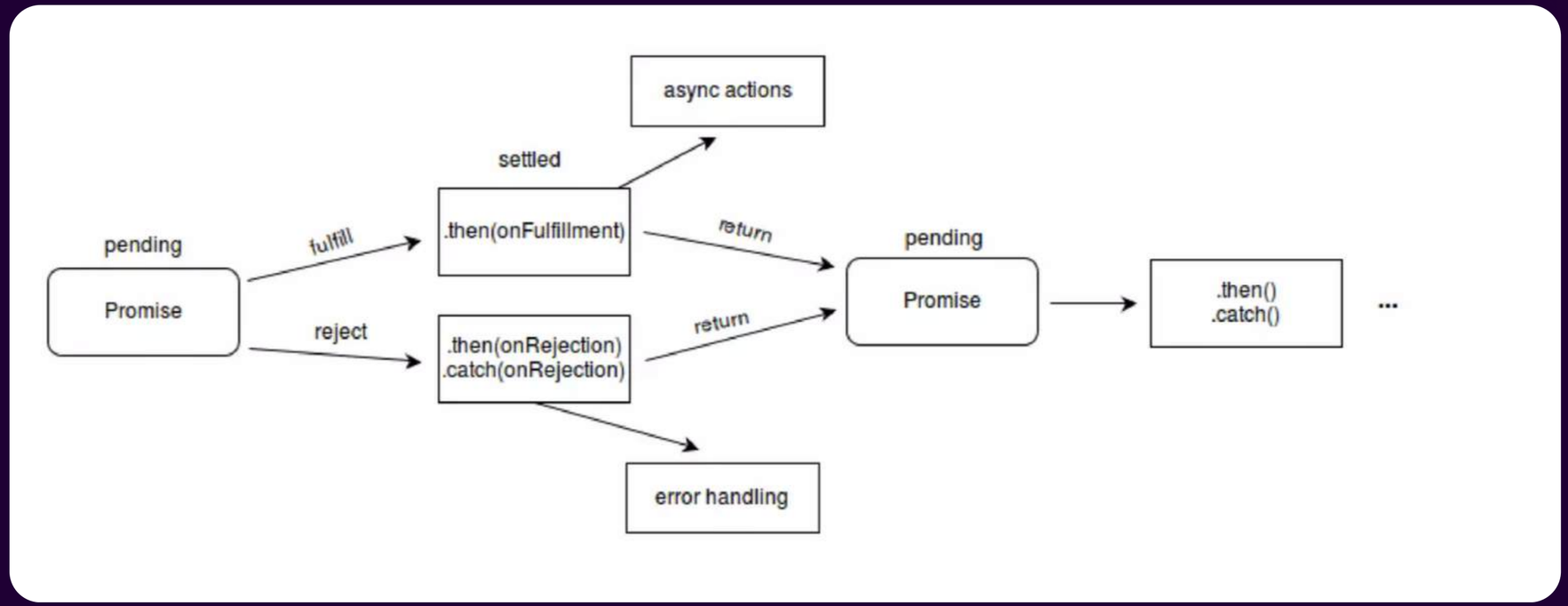
    }).then(() => {
        setTimeout(() => {
                document.getElementById("title4").style.visibility = "visible"
                resolve()
            }, 1000);
    }).catch((error) => {
        console.log("error from "+ error)
    })
```

THEN() ET CATCH()



CHAÎNAGE ET TRANSFORMATION DES PROMISES

Comme les méthodes then() et catch() retournent des promises, elles peuvent être chaînées.



PROMISE.ALL()

Avec Promise.all(), on peut exécuter plusieurs promesses en parallèle et attendre qu'elles soient toutes terminées.

```
const promesse1 = fetch("https://jsonplaceholder.typicode.com/posts/1").then(res => res.json());
const promesse2 = fetch("https://jsonplaceholder.typicode.com/posts/2").then(res => res.json());
💡
Promise.all([promesse1, promesse2])
    .then(resultats => console.log("Toutes les données :", resultats))
    .catch(erreur => console.error("Une erreur est survenue :", erreur));
```

PROMISE.RACE()

Retourne la première promesse qui se termine (résolue ou rejetée).

```
const promesseRapide = new Promise(resolve => setTimeout(() => resolve("Rapide !"), 1000));
const promesseLente = new Promise(resolve => setTimeout(() => resolve("Lente..."), 3000));

Promise.race([promesseRapide, promesseLente])
    .then(result => console.log("Résultat :", result))
    .catch(error => console.error("Erreur :", error));
```

LES AVANTAGES DES PROMISES EN JAVASCRIPT

01

ÉLIMINATION DU CALLBACK HELL

02

MEILLEURE GESTION DES ERREURS

03

CHAÎNAGE

04

CODE PLUS PRÉVISIBLE

LES INCONVÉNIENTS DES PROMISES EN JAVASCRIPT

01

COMPLEXITÉ INITIALE

02

PROBLÈMES DE PERFORMANCE POTENTIELS

03

MANQUE DE CONTRÔLE SUR L'ANNULATION

04

INCOMPATIBILITÉ AVEC CERTAINS SYSTÈMES ANCIENS

CONCLUSION



En conclusion, les Promises en JavaScript sont une solution puissante pour gérer l’asynchronisme de manière plus propre et structurée par rapport aux callbacks traditionnels. Elles permettent d’éliminer le "callback hell

YouCode
POWERED BY SIMPLON

MERCI

